WEEK -6

Minor Project (BTCS607N)-Weekly Progress Report

Project Title: Drone-Based Intelligent System for Apple Orchard Management in Himachal Pradesh.

Group ID: G9

Branch & Section: BTech CSE (AIML with Microsoft), AIDS -A

Date: 25-02-2026

1. Executive Summary

A. **Current Sprint:** Sprint 1 (Issue – 6)

B. **Sprint / Issue Goal:** Finalizing System Design and Integration for Apple Orchard Management Software.

C. **Status:** On Track

2. UML & OOAD Progress

A. **Diagrams Completed this week:**

1. **UML Class Diagram:**
   The class diagram has been finalized with the correct attributes, methods, and relationships between system components, such as Disease Detection, Health Classification, and Advisory System.
2. **UML Package Diagram:**
   The package diagram has been created to organize the system's classes into logical modules based on functionality.

B. **Diagrams Revised:**

1. **Class Diagram:** Revised the diagram as per system design, ensuring all components like HealthAnalysis, DiseaseDetection, and AdvisorySystem are included with appropriate methods, visibility, and interfaces.
2. **Package Diagram:** Updated to reflect the modularization of the system more accurately, ensuring that each package represents a logical group of related components.

c. **OO Concept Applied:**

SOLID Principles

1. Single Responsibility Principle (SRP):
   A class should have only one job or responsibility. For example, a User class handles user-related tasks like login, not image processing.

2. Open/Closed Principle (OCP):
   Classes should be open for extension but closed for modification, meaning you can add new functionality (like new disease types) without changing existing code.

3. Liskov Substitution Principle (LSP):
   Subtypes must be able to replace their base types without affecting system behavior. For instance, FungalDetection and ViralDetection should work the same when used as DiseaseDetection.

4. Interface Segregation Principle (ISP):
   Clients should only depend on the methods they need. For example, the IUser interface should only include user-related methods, not unrelated ones.

5. Dependency Inversion Principle (DIP):
   High-level modules should depend on abstractions, not concrete implementations. For example, the NotificationSystem should rely on the INotification interface, not specific notification types.

Design Patterns

1. Factory Pattern:
   Creates objects without specifying the exact class. For example, NotificationFactory creates different types of notifications.

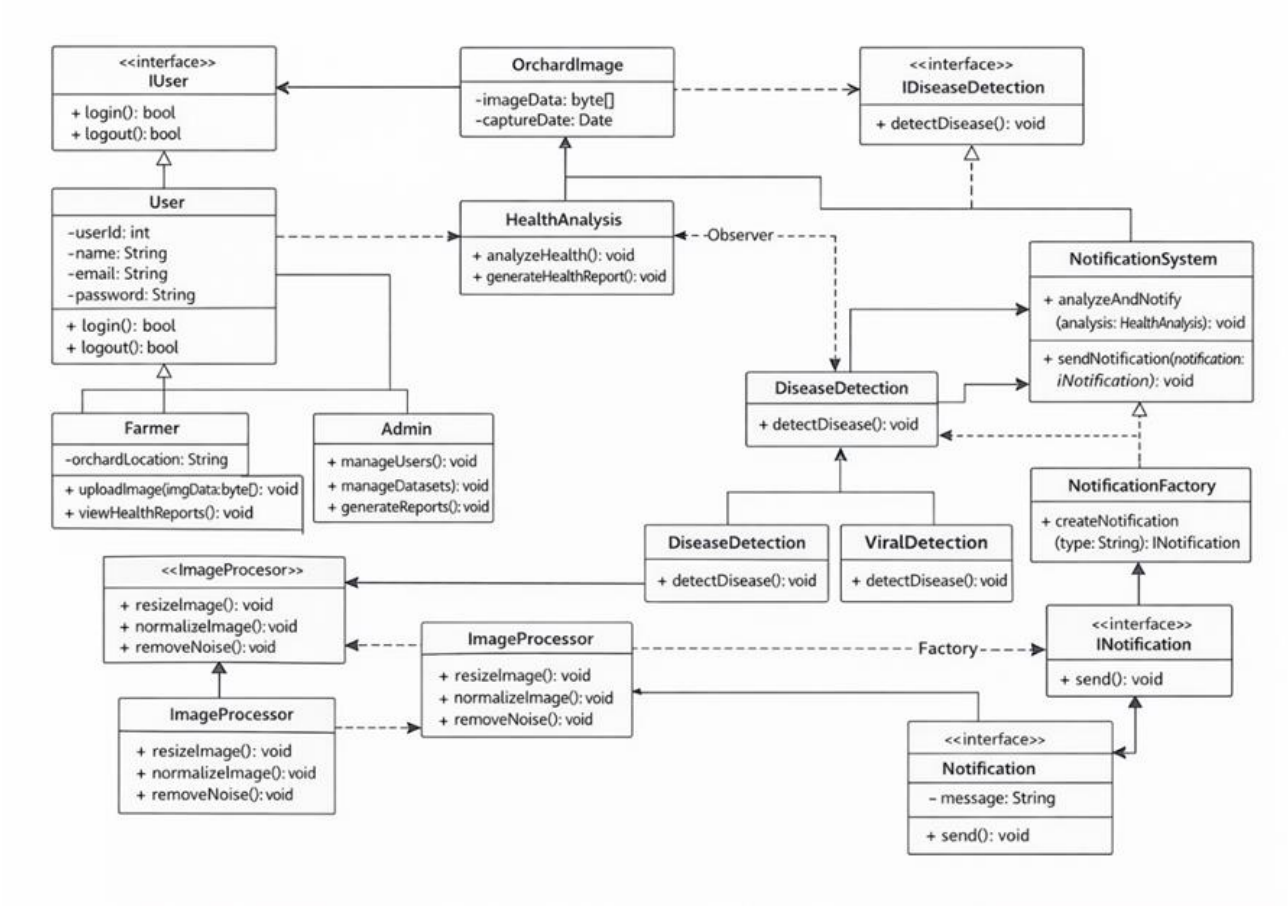
2. Strategy Pattern:
   Allows selecting an algorithm at runtime. For instance, different disease detection methods (FungalDetection, ViralDetection) can be chosen dynamically.

3. Observer Pattern:
   One object listens for changes in another. For example, HealthAnalysis observes OrchardImage and reacts when new images are uploaded.

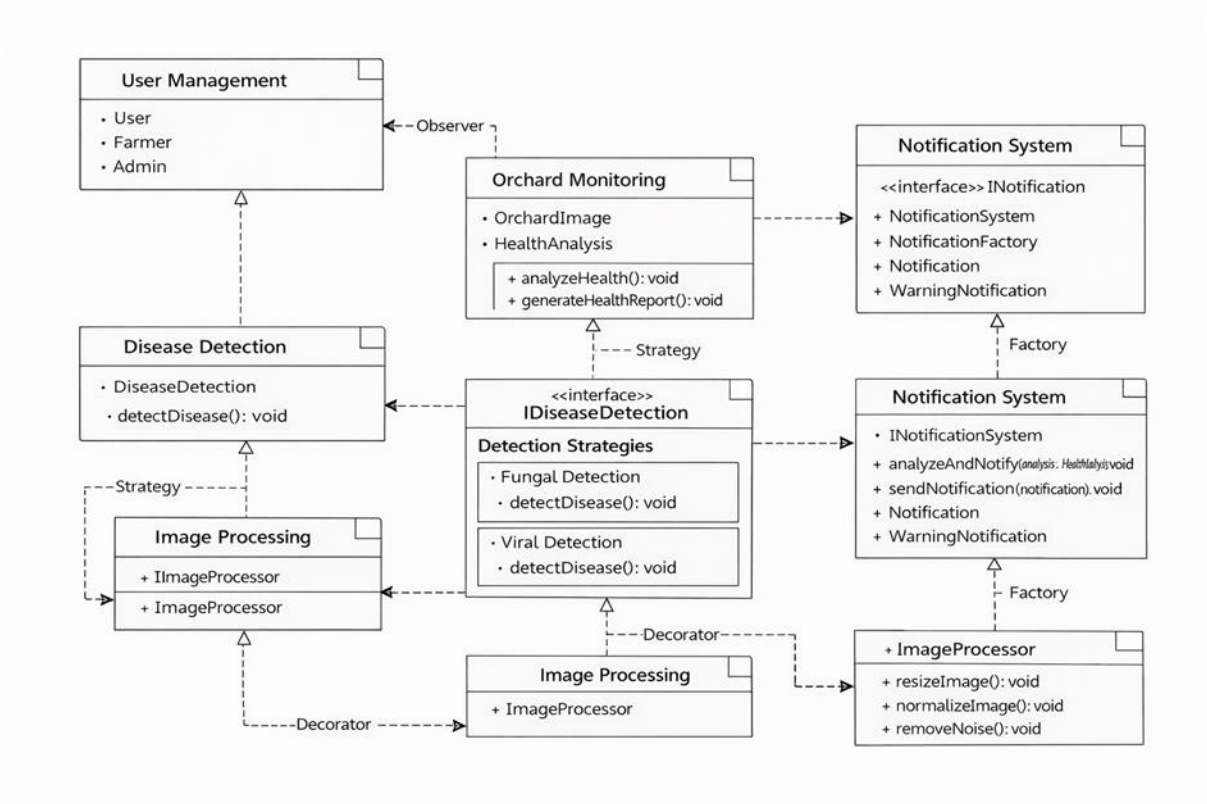
4. Decorator Pattern:
   Adds behavior to objects dynamically. For example, ImageProcessor can be extended to include image resizing or noise removal.

These principles and patterns help in building scalable, maintainable, and flexible systems.

Class Diagram with SOILD Principles and Design Patterns:



Package Diagram with SOILD Principles and Design Patterns:



3.Technical Execution (DevOps & Git)

A. **Jira Tasks Completed:**

1. Designed and validated the UML Class Diagram and Package Diagram.
2. Completed the Advisory System Development Plan.
3. Developed the Yield Estimation Development Plan.

Link://ayushshrivastava.atlassian.net/jira/software/projects/DBISFAOM/boards/2?atlOrigin=eyJpIjoiY2I0Y2FjMmQ2MTliNDBlM2FjYzEyN2U1MjdkY2QzNGMiLCJwIjoiaiJ9

B. **Git Activity:**

1. Active branches: main
2. Documentation updated with **Class Diagram** and **Package Diagram** details.

Link : https://github.com/yp1818/Drone-Based-Apple-Orchard-Management

C. **CI/CD Status:** Not applicable (design phase – no code yet)

4. Testing Summary

A. **Total Unit Tests:** Not applicable in design phase

B. **Pass Rate:** Not applicable in design phase

C. **Key Bug Identified:** No major issues were identified during this phase.

5. Goals for Next Week

System Architecture:

1. **UML Deliverables**:

   A. **Component Diagram**: Visualizing physical modules (API, Database, UI).

   B. **Deployment Diagram**: Mapping software components to hardware/nodes (Cloud, Servers).

Mentor Signature **Project Guide Signature**